

Root User V/S IAM User

In AWS, **root users** and **IAM users** serve different purposes and have distinct levels of access and control. Here's a breakdown of their key differences:

1. Root User (AWS Account Owner)

Creation: Created automatically when you first set up an AWS account.

Access Level:

- Has **full administrative privileges** over the entire AWS account.
- Can perform **all actions** on all resources and services.
- Can close the AWS account.

Usage:

- Should be used **only for critical account-level tasks** (e.g., changing account settings, enabling MFA, restoring IAM permissions).
- Should **not** be used for daily operations (to minimize security risks).

2. IAM User (Identity and Access Management User)

-Creation: Manually created by the root user or an admin IAM user.

Access Level:

- Has **no permissions by default** (must be explicitly granted via IAM policies).
- Can be assigned **fine-grained permissions** (e.g., read-only access to S3, full access to EC2).
- Cannot perform **account-level actions** (e.g., changing billing info, closing the account).

Usage:

- Used for **day-to-day operations** by individuals, applications, or services.
- Best practice is to follow the **principle of least privilege** (grant only necessary permissions).

Who Can Assign Permissions to IAM user?

- Root user (has full access by default).

-IAM users with sufficient permissions (e.g., `AdministratorAccess` or `IAMFullAccess`).

Access to IAM user by root user:

1. Sign in to the **AWS Management Console** as the **root user** or an **admin IAM user**.
2. Go to **IAM (Identity and Access Management)** → **Users**.
3. Select the **IAM user** you want to modify.
4. Under **Permissions**, click **Add permissions**.
5. Choose:
 - **Attach existing policies directly** (select AWS-managed or custom policies).
 - Or **Create a custom policy** (JSON-based).
6. Click **Next: Review** → **Add permissions**.

AWS as Host and Provider

AWS acts as both a host and a provider for Large Language Models (LLMs). For eg: it acts as host to Meta, Anthropic, Mistral etc, it is also called as provider because it has its own llm models like Titan.

Temperature vs Top P

Temperature controls the randomness of predictions by scaling the logits (probabilities) before sampling.

Higher values = more creative, diverse, but potentially less coherent outputs, Increases randomness.

Lower values = more deterministic, focused, and repetitive outputs, always picks the most likely next token.

Top-P is a way to control how "picky" the AI is when choosing the next word. Instead of considering every possible word, the AI only looks at the smallest group of likely words. Low Top-P (0.3)- Very picky; only the most obvious words are chosen since it is picking words from only top 30% of likely words. "The sky is blue." (Boring but safe). High Top-P (0.9)- More flexible; allows some creativity but avoids nonsense since it is picking words from top 90% of likely words. "The sky is a hazy shade of violet at dusk." (Nicely varied).

Boto

Boto is the official AWS SDK (Software Development Kit) for Python, allowing developers to interact with AWS services programmatically. Boto3 acts as a bridge between your local Python code (e.g., in VS Code) and AWS services in your cloud account.

AWS Bedrock

AWS Bedrock is a fully managed service that provides access to high-performing foundation models (FMs) from leading AI companies (like Anthropic, Meta, Mistral, etc.)

Key Features of AWS Bedrock

1. Access to Multiple Foundation Models

Host: Anthropic, Meta, Mistral, AI21 Labs

Provider: Amazon Titan

2. Serverless & Fully Managed

No infrastructure setup (AWS handles scaling, availability, updates).

Pay-per-use pricing (no upfront costs).

3. Fine-Tuning & Customization

Fine-tune models with your own data (private customization).

Retrieval-Augmented Generation (RAG) – Integrate external knowledge sources (e.g., company docs).

Example Workflow: Upload training data to Amazon S3> Fine-tune Llama 2 via Bedrock's console/API> Deploy the custom model.

4. Integration with AWS AI/ML Services

Amazon SageMaker: Train/deploy custom models.

AWS Lambda: Trigger Bedrock from serverless functions.

Amazon Kendra: Augment responses with enterprise data.

5. Agents for AI Assistants

Bedrock Agents: Build AI assistants that execute tasks (e.g., book flights, answer FAQs).

Orchestrates: interactions between FMs, Knowledge Bases, Guardrails, Code interpreter, and other AWS services.

Use Case: A customer support chatbot that fetches order details from Amazon DynamoDB.

6. Knowledge Bases (RAG)

Connect models to your data (PDFs, databases, wikis).

Reduces hallucinations by grounding responses in facts.

How it Works:

Upload documents to S3> Bedrock indexes them into a vector database (e.g., OpenSearch)> Queries fetch relevant snippets before generating answers.

7. Model Evaluation & Monitoring

Automatic evaluation of model performance.

CloudWatch Logs: Track API usage, errors.

Hands-on Tutorial: Deploying a Fine-Tuned Model on AWS Bedrock

Step 1: Prepare Your Training Data

AWS Bedrock requires training data in JSONL (JSON Lines) format for fine-tuning.

Example training data for a chatbot:

```
{"input": "What is the capital of France?", "output": "Paris"}  
{"input": "Who discovered gravity?", "output": "Isaac Newton"}  
{"input": "Translate 'Hello' to Spanish", "output": "Hola"}
```

- Save this file as training_data.jsonl.

- Upload it to an S3 bucket:

```
aws s3 cp training_data.jsonl s3://your-bucket-name/
```

Step 2: Fine-Tune the Model

1. Go to AWS Bedrock Console → Custom Models → Fine-tune Model.
2. Choose a foundation model (e.g., Amazon Titan).
3. Configure hyperparameters: • Epochs: 10, • Batch size: 1, • Learning rate: 0.00005
4. Select the S3 bucket where you uploaded training data.
5. Start the fine-tuning job and monitor logs in CloudWatch.

Step 3: Deploy the Fine-Tuned Model

Once fine-tuning is complete:

- Navigate to Bedrock Console → Deploy Model.
- Deploy the model as an API endpoint.

Step 4: Use the Model for Inference

You can now send requests to the model using AWS SDK (Python – Boto3):

Python Code to Invoke the Model

```
import boto3
```

```
import json
```

Initialize AWS Bedrock client

```
bedrock = boto3.client("bedrock-runtime", region_name="us-west-2")
```

Define input for inference

```
payload = { . "inputText": "What is the capital of Japan?" }
```

Convert payload to JSON

```
payload_json = json.dumps(payload)
```

Invoke the fine-tuned model

```
response = bedrock.invoke_model(  
    . modelId="your-custom-model-id",  
    . contentType="application/json",  
    . body=payload_json)
```

Read and print response

```
result = json.loads(response["body"].read())  
  
print(result)
```

Key Takeaways

- ✅ Fully managed service → No need to set up infrastructure.
- ✅ Custom fine-tuning → Train AI models with domain-specific knowledge.
- ✅ Easy deployment → Accessible via API.

Knowledge Base

1. S3 Bucket (Raw Storage) vs. Knowledge Base (RAG System)

Feature	S3 Bucket	AWS Bedrock Knowledge Base
Purpose	Stores files (PDFs, docs, etc.).	Turns files into searchable knowledge using AI.
Search Capability	No semantic search (only file/keyword).	Vector search (finds contextually relevant snippet).
AI Integration	None (just storage).	Connects to LLMs (e.g., Claude) for RAG .
Setup	Upload files manually.	Requires embeddings + vector store (e.g., OpenSe

Example Workflow:

1. Upload files to S3 (e.g., `s3://my-docs/company-policies.pdf`).
2. Create a **Bedrock Knowledge Base** linked to S3.
3. Bedrock:
 - Extracts text.
 - Generates embeddings (via Titan/Cohere).
 - Stores vectors in OpenSearch/Pinecone.
4. Now you can **query** it:
"What's the vacation policy?" → Returns answers grounded in your PDFs.

Guardrails

To prevent inappropriate, offensive, or harmful content, apply filters to ensure that the generated content adheres to the standards or guidelines that you have established.

Model Evaluation

Evaluation

Evaluation involves two key components: Model evaluation examines performance through automatic analysis or human input for aspects like brand voice, friendliness, and relevance, while Knowledge base evaluation (RAG evaluation) provides metrics including context relevance(retriever), correctness, and completeness using defined criteria and LLMs as judges for evaluation.

AWS Bedrock provides tools to evaluate, compare, and monitor foundation models (FMs) to ensure they meet accuracy, safety, and performance standards.

A. Automatic Model Evaluation (Pre-Built Metrics)

Bedrock automates evaluation for:

- Accuracy (e.g., Q&A correctness).
- Toxicity (harmful content detection).

- Robustness (consistency across prompts).
- Latency (response time).

B. Custom Evaluation (Bring Your Own Metrics)

- Define custom test datasets (e.g., domain-specific questions).
- Use AWS-provided templates or your own scripts.

Configure Metrics

- For **automatic evaluation**, select pre-defined metrics (accuracy, toxicity).
- For **custom evaluation**, upload a Python script to calculate scores.

AWS Lambda

AWS Lambda is a serverless, event-driven (eg, event like uploading file in s3 bucket trigger's code in lambda) **compute service that lets you run code without managing servers. It:**

- **Executes functions in response to triggers** (e.g., S3 uploads, API calls).
- **Auto-scales from zero to thousands of parallel invocations.**
- **Charges per millisecond of runtime (pay only when code runs).**

Interview-Friendly Example:

"Lambda is like a temporary worker who shows up only when needed (e.g., when a file is uploaded to S3), performs a quick task (like resizing an image), and leaves immediately after. No need to keep a full-time employee (server) on payroll 24/7."*

Real-Life Examples of creating lambda functions

- 1. Automatic Photo Resizing:** You upload a photo to Amazon S3 (cloud storage). Lambda triggers automatically, resizes the photo, and saves it. (Lambda requires code for this task)
- 2. Backend for a Website:** A user clicks a button on your website> API Gateway calls Lambda, which fetches data from a database> Lambda returns the result
- 3. Daily Email Reports:** Every morning at 7 AM, Lambda runs like an alarm clock> It fetches data, generates a report, and emails it to you.
- 4. Automated File Conversions:** Convert PDFs to text when uploaded to S3 using AWS Textract.
- 5. Chatbots:** Process user messages (e.g., Slack/WhatsApp) with Lex/Bedrock.
- 6. Database Cleanup:** Delete old records from DynamoDB nightly (scheduled trigger).
- 7. Authentication:** Validate user logins via Cognito triggers.

Why Use Lambda?

- **No servers:** AWS manages everything.
- **Pay per use:** Only pay when the robot works.
- **Super scalable:** Handles 1 user or 1 million users.
- **Lambda = A tiny, event-driven program** (eg, event like uploading file in s3 bucket trigger's code in lambda) that runs in the cloud.
- **Use it for small, automatic tasks** (e.g., process files, APIs, cron jobs).
- **No servers, no headaches!**

2. Server vs. Serverless

Server (EC2)	Serverless (Lambda)
You rent a virtual machine 24/7.	No VM—AWS runs your code on-demand.
You manage OS, updates, scaling.	Zero infrastructure management.
Pay hourly (even if idle).	Pay per execution (100ms increments).
Example: Running a website backend 24/7.	Example: Resizing photos only when uploaded.

Analogy:

Server = Owning a pizza oven (always there, even if unused).

Serverless = Ordering pizza on-demand (only pay when you eat).

Lambda can be connected to multiple AWS Services

Lambda can be triggered by 30+ AWS services. Common ones:

- S3: Process files after upload (e.g., resize images).
- API Gateway: Build REST APIs (backend for web/mobile apps).
- DynamoDB: Run code when data changes (e.g., send notifications).
- CloudWatch Events: Scheduled tasks (e.g., nightly backups).
- SNS/SQS: Process messages from queues/notifications.
- Kinesis: Analyze real-time data streams (e.g., clickstreams).
- Lex: Power chatbots with natural language processing.

Example Setup:

- S3 Trigger: Lambda runs when a new file lands in a bucket.
- DynamoDB Trigger: Lambda runs when a database record is updated.

Write Lambda> create function and write Code (Examples)

Example 1: Simple "Hello World"

```
def lambda_handler(event, context):

    print("Hello from Lambda!")

    return {

        'statusCode': 200,

        'body': 'Success!'

    }
```

Test it: Click "Test" → Use default test event → "Invoke".

Output: Hello from Lambda! appears in logs.

Understanding event and context in Lambda Functions (eg in def lambda_handler(event, context))

1) event Parameter

- **What it is:** A JSON-formatted **input data** passed to the Lambda function when triggered.
- **Where it comes from:**
 - a) **API Gateway:** Contains HTTP request details (headers, query parameters, body).

```
{
```

```
    "httpMethod": "GET",
```

```
    "queryStringParameters": {"city": "Paris"},
```

```
"body": "{}"

}
```

b) **S3 Trigger**: Includes bucket name and file key.

```
{

  "Records": [{

    "s3": {

      "bucket": {"name": "my-bucket"},

      "object": {"key": "image.jpg"}

    }

  ]

}
```

context Parameter

What it is: An object with **runtime information** (e.g., function name, remaining time).

*"Think of **context** like a dashboard showing your Lambda's current fuel (time left), speed (memory), and engine ID (request ID)."*

def lambda_handler(event, context):

print("Function name:", context.function_name)

print("Remaining time (ms):", context.get_remaining_time_in_millis())

print("AWS request ID:", context.aws_request_id)

Status Code and **body** in Lambda function

For Example when we create lambda function> in Code section, it will have code like

```
{  
  
    'statusCode': 200,  
  
    'body': 'Success!'  
  
}
```

What They Mean

statusCode: 200:

-An HTTP status code (200 = success, 404 = not found, 500 = server error).

-**This code only required for API Gateway-triggered Lambdas** (to format HTTP responses).

Is This code Common to All Lambda Functions?

-**No!** These fields are **only needed when Lambda is triggered by API Gateway**.

-**Other triggers (S3, DynamoDB, etc.) ignore these fields**—they expect no specific response format.

1. What Happens When You Click "Deploy" in Lambda?

Your code (inline editor changes or uploaded ZIP) is saved and deployed in the \$LATEST version of the Lambda function in Lambda's backend. Eg: You edit a Python function that processes API Gateway requests. Clicking Deploy and Test saves the code, runs it with a mock {"body": "test"} event, and shows the response (e.g., "Hello, test!").

2. Creating layers in Lambda code (eg: uploading boto3 zip files)

You can configure your Lambda function to use additional code and content in the form of

layers. A layer is a ZIP archive that contains libraries, a custom runtime, or other dependencies. With layers, you can use libraries in your function without needing to include them in your deployment package. If your Lambda function includes layers, Lambda extracts the layer contents into the /opt directory in the function execution environment.

3. How to Use the Test Section in AWS Lambda

The Test feature in AWS Lambda allows you to manually trigger/test your function (code) with custom input events (test data/test case) to verify its behavior before deploying it to production.

1. Accessing the Test Section

Open your Lambda function in the AWS Console> Click the Test tab (next to the "Code" tab)> If no test event exists, click "Configure test event".

2. Understanding the Event JSON Field

The Event JSON is a mock input(test data/test case) that simulates how your Lambda function is triggered by AWS services (e.g., API Gateway, S3, SQS).

Common mock input(test data/test case)Structures

A. API Gateway Event (HTTP Request)

```
{
  "body": "{\"name\":\"John\"}",
  "headers": {"Content-Type": "application/json"},
  "httpMethod": "POST",
  "path": "/test",
  "queryStringParameters": {"id": "123"}
}
```

- Used when Lambda is behind an **API Gateway REST/HTTP API**.
- Your function processes `event[ "body" ]`, `event[ "headers" ]`, etc.

B. S3 Event (File Upload)

```
{  
  "Records": [  
    {  
      "s3": {  
        "bucket": {"name": "my-bucket"},  
        "object": {"key": "example.jpg"}  
      }  
    }  
  ]  
}
```

- Simulates an S3 trigger (e.g., when a file is uploaded).
- Your function can read `event["Records"][0]["s3"]["bucket"]["name"]`.

Practical Example

Lambda Code (Python)

```
def lambda_handler(event, context):  
    name = event["queryStringParameters"]["name"]  
  
    return {  
        "statusCode": 200,  
        "body": f"Hello, {name}!"  
    }
```

Test Event/Test data (API Gateway Mock)

```
{
```

```
"queryStringParameters": {"name": "Alice"},  
  
"httpMethod": "GET"  
  
}
```

Then Test Output will be:

```
{  
  
  "statusCode": 200,  
  
  "body": "Hello, Alice!"  
  
}
```

4. Explain Monitor Section AWS lambda.

Monitoring in AWS Lambda refers to tracking the performance, errors, and resource usage of your functions using tools like:

- Amazon CloudWatch Logs (for logs)
- CloudWatch Metrics (for performance data)
- AWS X-Ray (for tracing requests)
- Lambda Insights (advanced monitoring)

Once you use the api or get any other output using the code in the code section, just click on 'cloudwatchlog' in the monitor section, it will display all the output logs.

5. What is Monitor in lambda?

=> Goto Monitor> click on General Configurations to edit the fields below.

B. Resource Allocation

Setting	Description	Impact
Memory (MB)	RAM (128 MB – 10,240 MB).	Higher memory = more CPU.
Ephemeral Storage	<code>/tmp</code> disk space (512 MB – 10 GB).	Useful for file processing.
Timeout	Max runtime (1 sec – 15 min).	Set to ~2x expected duration.

=> Goto Monitor> click on 'Permissions'> click on the link below Rolename to update permissions like administrator, bedrock permissions etc.

6. What is Function url in lambda> configurations?

AWS Lambda Function URLs provide a dedicated HTTP(S) endpoint to invoke your Lambda function directly, without needing API Gateway, Application Load Balancer (ALB), or other intermediaries. It's a simple way to expose a Lambda function as a webhook, microservice, or backend API.

How to Configure a Function URL

- > Go to Lambda Console → Select your function → Configuration tab → Function URL.
- > Click "Create function URL".
- > Choose auth type: AWS_IAM or NONE.
- > Configure CORS if needed (e.g., allow * for public access).
- > Copy the URL and test it (e.g., with curl or Postman).

Amazon S3 Bucket

Amazon S3 (Simple Storage Service) is a scalable object storage solution for storing and retrieving any amount of data.

Feature	Use Case
Unlimited Storage	Store any amount of data (files from 0B to 5TB).
High Durability (99.999999999%)	Data is replicated across multiple AZs.
Low Latency	Fetch objects in milliseconds.
Security & Compliance	IAM policies, encryption, audit logging.
Static Website Hosting	Host HTML/CSS/JS files directly from S3.
Event Notifications	Trigger Lambda, SQS, or SNS on file uploads.
Access Control (ACLs & Bucket Policies)	Define who can read/write objects.

AWS API Gateway

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale.

Its features include:

1. API Creation & Deployment

- RESTful APIs: Create REST APIs to expose HTTP endpoints.
- WebSocket APIs: Build real-time, two-way communication APIs (e.g., chat apps, live updates).
- HTTP APIs: Low-latency, cost-effective APIs for proxy integrations (Lambda, HTTP backends).

2. Integration with AWS Services

- Lambda Functions: Invoke serverless functions directly.
- EC2 & ECS: Integrate with backend servers running on AWS.
- DynamoDB: Perform CRUD operations via API Gateway.
- SQS/SNS/Kinesis: Send data to AWS messaging/streaming services.
- Step Functions: Orchestrate workflows via API calls.

1. HTTP API (Fast Food 🍔)

Definition: A lightweight, low-cost API designed for quick communication between clients and servers. It follows basic HTTP protocols (GET, POST, etc.) but doesn't enforce strict rules like REST.

This api type is best for High-speed, low-cost needs (e.g., serverless apps).

You can think of the HTTP API as a basic box where you can quickly put toys in (requests) and take toys out (responses). It is used when you just need to send and receive stuff fast without extra steps. Eg- You ask AWS Lambda: `*"What's 2+2?"*` → Lambda replies `"4"` → API Gateway sends it back. Done!

- **Features:**

-  **Cheaper** (~3x cheaper than REST APIs).
-  **Faster** (lower latency).
-  Supports **JWT auth** (for simple security).
-  **No advanced features** like caching, request transformations, or API keys.

Note: while creating integrations we will integrate it with the lambda function that we created.

Integrations Info:

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Integrations connect routes to backend resources. You can add integrations now, or you can create integrations after you create your API. To create routes in the next step, you must add integrations here. You can create integrations with Lambda functions or public HTTP endpoints while you create your API. To create integrations with private resources in a VPC or AWS services, first create your API, and then create those integrations.

API Gateway > APIs > Create API > Create HTTP API

Step 1
Configure API

Step 2 - optional
Configure routes

Step 3 - optional
Define stages

Step 4
Review and create

Configure routes - optional

Configure routes [Info](#)

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

Method	Resource path	Integration target	
P...	/agentbedrockone	agentbedrockone	Remove

Add route

Cancel

Review and create

Previous

Next

In api gateway, after creating http api, why do we create stages?

When you create an HTTP API (or REST API) in AWS API Gateway, stages act like separate "environments" for your API. Think of them as different versions of your API that serve specific purposes (e.g., testing, production).

Example:

dev stage → For developers testing new features.

prod stage → Live version for real users.

Prevents accidental changes to the live API.

Each stage has its own unique URL

What is the automatic deploy toggle button that we get while creating stages in http api gateway?

What is Automatic Deployment?

When ON (Enabled):

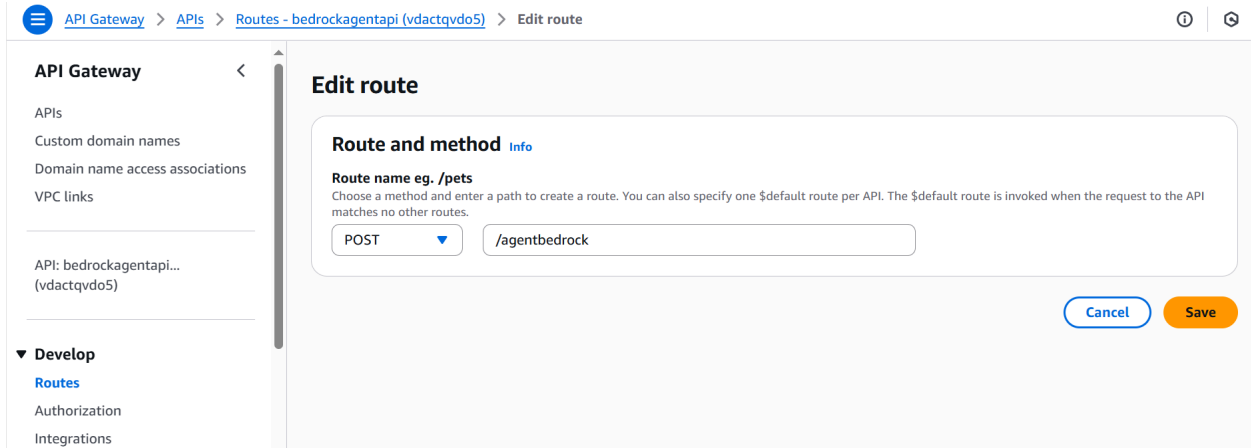
Any changes you make to the API (e.g., routes (as shown in below image), integrations) automatically deploy to this stage without manual action.

Example: If you add a new route (/users), it instantly becomes live in the stage.

When OFF (Disabled):

You must manually deploy changes to the stage (like clicking a "Deploy" button).

Example: You modify the API, but it won't go live until you explicitly deploy it.



2. RESTful API (Fine Dining 🍷)

A fully customizable API that follows REST principles (CRUD, resource-based). It is Best for Complex APIs needing security, caching, or monetization.

Think of Restful api as a smarter toy box with rules, labels, and security checks. Good to use this when you need control (like checking who can take toys, or caching responses).

Example:

You ask: "Give me user #123's data."

REST API checks permissions → transforms the request → caches the response → replies.

- **Features:**

- ☒ **Advanced security** (IAM, Cognito, custom Lambda authorizers).
- ☒ **Request/response transformations** (modify data before sending).
- ☒ **API caching** (improves performance).
- ☒ **Usage plans & API keys** (charge users per call).
- ☒ **More expensive & slower** than HTTP APIs (due to extra features).

Other Features of Restful API:

Caching: Storing responses temporarily to avoid repeated processing. Eg- If the answer was written 5 mins ago, it reuses it instead of asking the weather service again.

Monetization: Charging users for API access. For Eg: Free tier: 100 requests/day and Pro tier (\$10/month): 10,000 requests/day

Security: Controlling who can use your API.

Eg: A bank API checks:

- IAM Roles: Is the caller an AWS service?
- Lambda Authorizer: Does the user's token have "admin" access?
- JWT/OAuth: Is the login token valid?

API Keys: Unique passwords for tracking/limiting API users. For Eg- You give API_KEY=123abc to a mobile app → AWS tracks how many calls it makes. If the app exceeds 1000 calls/day, it's blocked.

Customizability, Rate-Limiting & Validation:

Customizable REST APIs

- Define URL paths (/orders, /users).
- Set HTTP methods (GET, POST, DELETE).
- Add middleware (e.g., "Check payment before creating an order").

Rate-Limiting: Controlling how often users can call your API. Free users: 10 requests/minute → Beyond that, they get 429 Too Many Requests. Paid users: 1000 requests/minute.

Request Validation: Checking if inputs are correct before processing.

Input= { "email": "invalid-email", "age": "twenty" }

API Rejects: { "error": "Email must be valid, age must be a number" }

CRUD: CRUD stands for the 4 basic operations you can perform on data in any application.

CRUD Operation	HTTP Method	Example Request (REST API)	What It Does
Create	POST	POST /users	Adds a new user
Read	GET	GET /users/123	Fetches user #123
Update	PUT/PATCH	PUT /users/123	Replaces user #123's data
Delete	DELETE	DELETE /users/123	Removes user #123

Think of CRUD like a **notebook**:

- **POST** = Writing a new page.
- **GET** = Reading a page.
- **PUT/PATCH** = Editing a page (rewriting vs. correcting).
- **DELETE** = Tearing out a page.

Example

- **HTTP API:** A weather app fetching data from Lambda (simple, fast).
- **REST API:** A banking API with auth, rate-limiting, and request validation.

Key Differences (Interview Ready 🎯)

Feature	HTTP API	RESTful API
Purpose	Simple, fast communication	Structured, resource-based interactions
Complexity	Minimal (just HTTP calls)	Follows REST principles (stateless, CRUD)
Caching	❌ No	✅ Yes (e.g., cache user data for 5 mins)
Security	Basic (API keys, JWT)	Advanced (IAM, OAuth, Lambda authorizers)
Use Case	Serverless apps, proxies	Complex apps (e.g., e-commerce, social media)
Cost (AWS)	\$1.00 per million calls	\$3.50 per million calls

AWS Sagemaker AI

Amazon SageMaker AI is a fully managed machine learning (ML) service. With SageMaker AI, data scientists and developers can quickly and confidently build, train, and deploy ML models into a production-ready hosted environment. It provides a UI experience for running ML workflows that makes SageMaker AI ML tools available across multiple integrated development environments (IDEs).

With SageMaker AI, you can store and share your data without having to build and manage your own servers. This gives you or your organizations more time to collaboratively build and develop your ML workflow, and do it sooner.

SageMaker AI provides managed ML algorithms to run efficiently against extremely large data in a distributed environment. With built-in support for bring-your-own-algorithms and frameworks, SageMaker AI offers flexible distributed training options that adjust to your specific workflows. Within a few steps, you can deploy a model into a secure and scalable environment from the SageMaker AI console.

Amazon Sagemaker Studio

Amazon SageMaker Unified Studio provides an integrated experience to use all your data and tools for analytics and AI. You can use Amazon SageMaker Unified Studio to discover your data and put it to work using familiar AWS analytics and machine learning services for model development, generative AI, big data processing, and SQL analytics, assisted by Amazon Q Developer.

You can also use Amazon SageMaker Unified Studio to work across compute resources using unified:

- notebooks,
- discover and query diverse data sources with a built-in SQL editor,
- train and deploy AI models at scale, and
- rapidly build custom generative AI applications

Amazon SageMaker Unified Studio also offers the following capabilities:

- Use all your data and tools in a single development environment
- Build and scale generative AI applications with Amazon Bedrock
- Gain insights with the most price-performant SQL engine from Amazon Redshift
- Unify data access across Amazon S3 data lakes, Amazon Redshift, and federated data sources with Amazon SageMaker Lakehouse
- Build, train, and deploy machine learning and foundation models, with fully managed infrastructure, tools, and workflows from Amazon SageMaker AI
- Prepare, integrate, and orchestrate data for analytics and AI at petabyte scale with Amazon EMR, Amazon Athena, and AWS Glue
- Discover, govern, and collaborate on data and AI securely, with a unified catalog, built on Amazon DataZone.

Amazon SageMaker Notebook

Amazon SageMaker **Notebook Instances** support a wide range of **programming languages, tools, frameworks, and platforms** for machine learning (ML) and data science.

Category	Supported Tools
Languages	Python, R, PySpark, SQL, Julia
Frameworks	TensorFlow, PyTorch, Scikit-learn, Hugging Face
AWS Services	S3, Glue, Lambda, SageMaker SDK
IDEs	JupyterLab, Jupyter Notebook
MLOps	SageMaker Pipelines, MLflow, Kubeflow

Dockers

In Docker, a "lambda image" typically refers to a Docker container image that is optimized for running AWS Lambda functions. A Lambda image in Docker is simply a container image built to run on AWS Lambda, following AWS's specifications. AWS Lambda traditionally allows you to upload code directly (e.g., as a .zip file), but it also supports running functions from Docker container images. These images must follow AWS Lambda's specifications to work correctly.

How It Works:

> Instead of uploading code directly to Lambda, you:

1. Build a Docker image that includes:

-Your function code.

-The Lambda Runtime Interface Client (RIC) (to handle Lambda invocations).

-Any required dependencies.

2. Push the image to Amazon ECR (Elastic Container Registry).

3. Configure Lambda to use this image.

Example Dockerfile for Lambda (Python)

```
FROM
```

amazon/aws-lambda-python:3.9- this is official AWS-provided base image for Python 3.9

COPY lambda_function.py \${LAMBDA_TASK_ROOT}:

Copies the Lambda function code into the container.

lambda_function.py is the Python script containing your Lambda function.

\${LAMBDA_TASK_ROOT} is an environment variable set by AWS Lambda (default: /var/task). This is where Lambda expects your function code to reside.

COPY requirements.txt and RUN pip install -r requirements.txt: Copies the requirements.txt file (listing Python dependencies) into the container and Installs the Python dependencies listed.

CMD ["lambda_function.handler"]: Defines the entry point for the Lambda function.
"""

FROM amazon/aws-lambda-python:3.9

Copy function code

COPY lambda_function.py \${LAMBDA_TASK_ROOT}

Install dependencies (if any)

COPY requirements.txt .

RUN pip install -r requirements.txt

Set the handler

CMD ["lambda_function.handler"]

